

WebRTC Mobile POC - Windows setup

Prepared for Shubham Sinha | 1 October 2026

1. What this package does

A two-person React Native CLI Android app with your web POC's room create/join flow, video/audio, private invite, chat, mic/camera controls, screen sharing with camera still visible, recording, and reconnect. The app adds front/rear camera switching and speaker/earpiece selection. Layout stacks on phones and expands on tablets/landscape; chat and controls scroll rather than overlap.

No LiveKit, Agora or Twilio account is needed. You run the Node signaling/recording server yourself. A public Google STUN endpoint is configured by default to discover network addresses; it is not a managed calling platform. You can change/remove STUN and configure your own coturn relay.

Recording runs on your PC/server, using headless Chrome. It receives both participants' microphone/camera/screen tracks and creates a WebM download. It is NOT device-only recording. Both participants must agree; revoking consent stops recording. Screen/system audio is not included. Android screen sharing is implemented; iOS is a scaffold, not feature-complete.

Verification: JavaScript bundles and server tests passed. Native APK build and real-device/media recording tests still need your local verification. See VERIFICATION.md.

2. Install prerequisites

Use Windows 10/11, Node 22.x (22.12 or newer), npm, JDK 17, Android Studio and a real Android phone (Android 7/API 24 or newer). Android 14+ is useful for testing modern screen-sharing permissions. A real phone is preferable to emulator camera/audio behavior. Start with two phones or one phone plus your PC browser.

In Android Studio > SDK Manager, install Android SDK Platform 36. Under SDK Tools > Show Package Details, install Android SDK Build-Tools 36.0.0, Platform-Tools, Command-line Tools, NDK (Side by side) 27.1.12297006 and CMake 3.22.1. Accept SDK licenses. Keep Gradle/AGP/Kotlin versions supplied with this project.

Set Windows environment variables in System > Advanced system settings > Environment Variables:

- JAVA_HOME: your actual JDK 17 installation folder (not its bin subfolder).
- ANDROID_HOME: normally C:\Users\YOUR_WINDOWS_USER\AppData\Local\Android\Sdk.
- Add %JAVA_HOME%\bin, %ANDROID_HOME%\platform-tools and %ANDROID_HOME%\emulator to Path.

Open a NEW PowerShell terminal and check:

```
node -v
npm -v
java -version
adb version
```

If Android SDK location is not detected, create mobile\android\local.properties with your actual path, using forward slashes:

```
sdk.dir=C:/Users/YOUR_WINDOWS_USER/AppData/Local/Android/Sdk
```

Do not commit local.properties. No global React Native CLI installation is required.

3. Extract and install

Extract the ZIP so that C:\WebRTC_Mobile_POC contains mobile, server-web, docs and Setup.cmd. Avoid OneDrive and very long directory names.

Easiest: double-click Setup.cmd, or run it from PowerShell. It installs both projects, creates server-web\.env only if absent, and builds the browser companion. npm may take several minutes; Puppeteer downloads Chrome for recording. Keep internet available during dependency setup.

Equivalent manual commands:

```
cd C:\WebRTC_Mobile_POC\server-web
Copy-Item .env.example .env
npm ci
npm run build
cd ..\mobile
npm ci
```

Run Copy-Item only on the first setup so you do not overwrite an edited .env. If npm is blocked as npm.ps1, use npm.cmd in the same commands. If Chrome was not downloaded, run npx puppeteer browsers install chrome in server-web, or set CHROME_PATH in .env to an installed Chrome executable. The optional path is not needed for a normal install.

The two package-lock.json files are included. Use npm ci to retain the chosen versions; do not upgrade all dependencies as a setup step.

4. Start the server

Terminal 1:

```
cd C:\WebRTC_Mobile_POC\server-web
npm start
```

Keep this terminal open. Alternatively double-click StartServer.cmd. The .env example sets HOST=0.0.0.0 and PORT=3001 so your phone can reach the PC. Open these on the PC:

- <http://localhost:3001/api/health> - expect JSON containing ok: true.
- <http://localhost:3001> - browser companion.

Allow Node.js through Windows Defender Firewall on your private/home network. Do not disable the firewall. If an explicit inbound rule is needed, allow TCP 3001 on the Private profile for testing. PC-side Chrome and the recorder also need local UDP connectivity for WebRTC.

Run ipconfig and find the IPv4 address of the Wi-Fi adapter, for example 192.168.1.10. On the phone's browser, open <http://192.168.1.10:3001/api/health>. If this fails, fix networking before attempting the call. Guest Wi-Fi/client isolation, VPNs and a Public firewall profile can block access.

5. Install and run the Android app

On the phone, enable Developer options and USB debugging. Connect USB and accept the RSA authorization prompt. Run:

```
adb devices
```

The phone must say device, not unauthorized. Terminal 2 starts Metro:

```
cd C:\WebRTC_Mobile_POC\mobile
npm start
```

Terminal 3 builds and installs the app:

```
cd C:\WebRTC_Mobile_POC\mobile
adb reverse tcp:8081 tcp:8081
npm run android
```

Keep Metro and the server running. The first Gradle/NDK download and native build can take time. If several devices are connected, use `npx react-native run-android --deviceId YOUR_ADB_SERIAL`. To configure reverse for a specific phone, put `-s YOUR_ADB_SERIAL` immediately after `adb`.

Set Server URL inside the app as follows:

Test mode	Server URL in app
Physical phone, same Wi-Fi	<code>http://192.168.1.10:3001</code> (replace with your PC IPv4)
Standard Android Studio emulator	<code>http://10.0.2.2:3001</code>
USB with adb reverse for 3001	<code>http://127.0.0.1:3001</code>
Hosted server	<code>https://your-domain.example</code>

For the USB server route, also run `adb reverse tcp:3001 tcp:3001`. This forwards signaling/HTTP, not WebRTC media: the two participants still need a working direct network route or TURN. On a physical phone, `localhost` normally means the PHONE, not your PC.

6. Make the first call

For phone-to-PC, first open `http://localhost:3001` in Chrome/Edge on the PC. Enter your name, allow recording if wanted, click Create a room, and allow camera/mic access. Copy the complete invitation link.

On Android, set Server URL to your PC's Wi-Fi IP (or the USB route above), enter the second participant's name, paste the complete link, and tap Join room. The mobile app deliberately uses the explicit Server URL field so a `localhost` invite can join the same server from a phone. Allow microphone, camera and notification prompts. The call should show Connected and both participants.

For two phones, set both to the same PC server URL. Create a room on one, Share invite, then paste it on the second and join. The POC rejects a third participant. Use headphones or separate rooms to prevent acoustic feedback during testing.

If a phone creates the room and the PC receives an `http://192.168...` invite, replace ONLY the address before `/#` with `http://localhost:3001` when opening it on that same PC. Keep `#room=...&key=...` unchanged. Desktop browser capture needs `localhost` or `HTTPS`; ordinary LAN HTTP is not a secure browser capture context. The native app can use LAN HTTP in the debug/POC build.

Rooms live in server memory and expire after two hours, or after ten minutes empty. A server restart removes rooms and chat history. Room keys are private invitation credentials; do not post them publicly.

7. Controls and recording

Mute mic stops sending microphone audio. Camera off releases camera capture; Retry camera opens it again. Flip camera switches front/rear. Use speaker / Use earpiece changes the requested output route; wired/Bluetooth devices and OEM policies can affect the actual route.

Share screen opens Android's system prompt. Select the screen/app to capture and confirm. Camera and screen use separate video tracks. Stop using the app control or Android's projection notification. Do not test protected/DRM content; Android may show it as black. Screen sharing sends video only, not device audio.

While showing your own app, a recursive mirror is expected; share another app for a useful test.

Both people must enable Allow server recording, and the call must be connected. Tap Record call, wait for the recording banner to change from connecting to recording, speak from both sides, optionally share a screen, then tap Stop recording. Wait for the download button BEFORE leaving. Either participant can stop; withdrawing consent or leaving also stops the recorder. Other still-connected participants receive the completed download link.

Downloads open in your phone browser. Files live in server-web\recordings for 24 hours by default; cleanup runs every minute. RECORDING_RETENTION_HOURS changes this. Download tokens are in server memory, so links stop working after a server restart even if the file remains on disk. Manual recovery of those files is available to the server owner. Limits: 15 minutes or 150 MB per recording, two concurrent server recorders. Camera, screen, network and CPU quality depend on hardware. Recording adds upload bandwidth from each participant to the server.

Background behavior: without screen sharing, this POC ends the call when you background the app. With screen sharing, the foreground call/projection services support continuing while switching apps. Lock-screen, battery optimization and device-specific capture behavior still require testing. This is not a production incoming-call service.

8. Build a standalone POC APK

This removes the Metro requirement, but the Node server must still run. The poc build type embeds JavaScript, uses the included development signing key and permits local HTTP for testing. It is NOT an app-store build.

```
cd C:\WebRTC_Mobile_POC\mobile\android
.\gradlew.bat assemblePoc
adb install -r .\app\build\outputs\apk\poc\app-poc.apk
```

You can copy that APK to another phone and install it after permitting installs from the chosen file manager. Both apps still need the same reachable server URL. The ZIP contains source code, not a prebuilt or device-verified APK.

The normal release build should use HTTPS and your own signing key before publishing. This POC genuinely uses camera, microphone and media-projection foreground services; do not remove those permissions while retaining the features. Play Console declarations and release hardening are separate work.

9. Calls across different networks

STUN alone does not guarantee connectivity through mobile carriers, corporate firewalls or restrictive NATs. Run coturn on a publicly reachable Linux server with a public IP. The example configuration is in server-web/deploy/turnserver.conf.example. Use a new random shared secret in coturn and server .env:

```
TURN_URLS=turn:YOUR_PUBLIC_IP:3478?transport=udp,turn:YOUR_PUBLIC_IP:3478?transport=tcp
TURN_SECRET=YOUR_RANDOM_SHARED_SECRET
ICE_TRANSPORT_POLICY=all
```

Configure coturn's external/public IP when behind NAT and open its listening and relay port ranges in the server firewall/cloud security group. Use the SAME secret for coturn's static-auth-secret and Node TURN_SECRET. Restart Node, create a fresh room, then test Wi-Fi versus mobile data. Credentials are generated per socket and the shared secret is not shipped in the app.

Temporarily set ICE_TRANSPORT_POLICY=relay to prove TURN is working; restart and rejoin, then confirm the connection reports TURN relay. Restore all afterward if you want direct connections when available. Host signaling/browser access behind HTTPS and set ALLOWED_ORIGINS to the exact browser origin(s). Coturn media needs separate network ports; an HTTP reverse proxy alone does not relay WebRTC packets. The recorder also needs media access to both participants.

10. Troubleshooting

Symptom	Check
Network request failed / server unreachable	Test /api/health from phone; check PC IP, HOST, Wi-Fi, VPN and Private firewall rules.
Signaling connects but video stays connecting	Same-network routing, UDP firewall, TURN configuration; test relay mode.
Browser camera missing on a LAN URL	Use localhost on the server PC or a trusted HTTPS hostname.
Metro bundle unavailable	Keep npm start running; repeat adb reverse for 8081 after reconnecting USB.
Camera denied / camera busy	Enable permissions in Android app settings; close other camera apps; tap Retry camera.
SDK location not found	Set ANDROID_HOME or mobile/android/local.properties to your real SDK path.
Gradle download timed out	Retry on a stable connection; wrapper timeout is 120 seconds. Keep the supplied Gradle version.
NDK/CMake missing	Install the exact NDK and CMake versions through SDK Manager.
Cannot find Chrome / recorder fails to start	In server-web run npx puppeteer browsers install chrome, or configure CHROME_PATH and restart.
Recorder media did not connect	Verify both clients are from this package, both consented, and server media routing/TURN works.
Screen-share permission canceled	Tap Share screen again and accept Android's system dialog.
Invalid/expired room	Server restarted, room expired or link incomplete; create a new room.
Download missing after leaving	Stop recording and wait for the file before leaving. Server disk may hold the saved file.

For native build diagnostics, run `npx react-native doctor` in mobile. Capture the FIRST failing Gradle task and error, not only the final BUILD FAILED line. Keep server console output and `adb logcat` when investigating device-specific behavior.

11. Acceptance checklist and platform scope

Before treating this POC as ready, test the following on your own Android devices:

- Install/open; deny camera and verify audio-only join; grant/retry camera.
- Phone-to-PC and two-phone calls: both voices and cameras, front/rear switch, mute/unmute and speaker route.
- Rotate the phone, open keyboard/chat, and verify controls on a small screen/tablet.
- Share screen while camera remains visible; stop from both app and OS prompt/notification.
- Both participants consent, record both voices and shared screen, stop, download and PLAY the WebM to verify actual sound/video.
- Revoke consent, leave while recording, lose network and rejoin, and test a third participant is rejected.

- Two different networks with TURN forced to relay; test recorder media too.
- Build/install the standalone APK without Metro and repeat core flows.

The included iOS folder can be a starting point on macOS with Xcode/CocoaPods, but it has not been built. Full iOS screen broadcasting needs a ReplayKit Broadcast Upload Extension, app-group/signing configuration and device validation. This package does not claim that implementation.

Automated commands:

```
cd C:\WebRTC_Mobile_POC\server-web
npm test
npm run build
npx playwright install chromium
npm run test:browser
cd ..\mobile
New-Item -ItemType Directory -Force build
npm run bundle:android
```

The browser test uses fake camera/mic devices and a synthetic screen; it does not replace Android testing. See VERIFICATION.md for this delivery's actual results.

12. Source and implementation references

- React Native environment setup: <https://reactnative.dev/docs/0.81/set-up-your-environment>
- Native WebRTC Android instructions: <https://github.com/react-native-webrtc/react-native-webrtc/blob/master/Documentation/AndroidInstallation.md>
- Native WebRTC iOS guidance: <https://react-native-webrtc.github.io/handbook/guides/extra-steps/ios.html>
- Puppeteer: <https://pptr.dev/>
- Your original WebRTC_POC.zip version 2 and WebRTC_POC_Technology_Stack.pdf supplied the web features and signaling baseline.

The versions here are pinned for this POC, not represented as the newest releases. User-specific preferences and project decisions are saved in PROJECT_CONTEXT.md for continuation.